NSSM - the Non-Sucking Service Manager

Usage

No "installation" of *nssm* is needed. Just place it somewhere on the system (preferably somewhere in your PATH) and run it.

Note however that *nssm* registers itself as an Event Log message source which means that running multiple instances or different version of *nssm* from different locations may cause confusion. Also note that if you run the Event Viewer it will open the *nssm* executable, preventing you from overwriting it. Keep this in mind if you come to upgrade *nssm*.

Some features are labelled as supported as of a particular version. If the version described is newer than that available from the download page there may be a pre-release build with the feature enabled.

Equivalent command examples below show the commands which would configure an *existing* service to match the screenshots. In many cases they represent the defaults for the parameters in question and are thus redundant. Any parameter can also be reset to its default value with

```
nssm reset <servicename> <parameter>
```

Installing a service

You can use *nssm* to install a service. The command to type is:

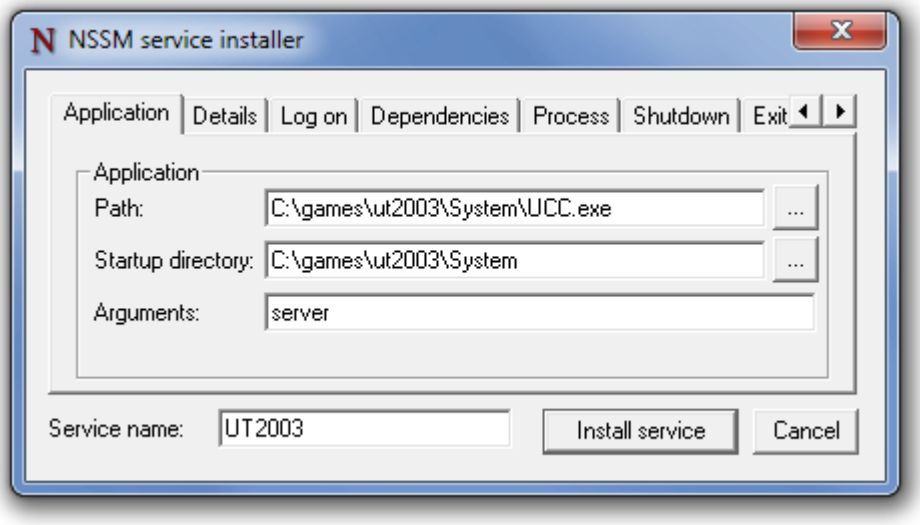
```
nssm install <servicename>
```

The installer consists of several tabs with lots of configurable parameters. Most are preset to *nssm*'s defaults, so it's possible to install a service without leaving the *Application* tab.

Application tab

The *Path* to the application (or script) you want to run and is the only mandatory field. If the application needs to start in a particular directory you can enter it in the *Startup directory* field. If the field is left blank the default startup directory will be the directory containing the application. The *Arguments* field can be used to specify any commandline arguments to pass to the application.

The screenshot below shows installation of a UT2003 server. The command to run such a service is **ucc server** so the full path to **UCC.exe** is entered under *Path* and **server** is entered under *Options*.



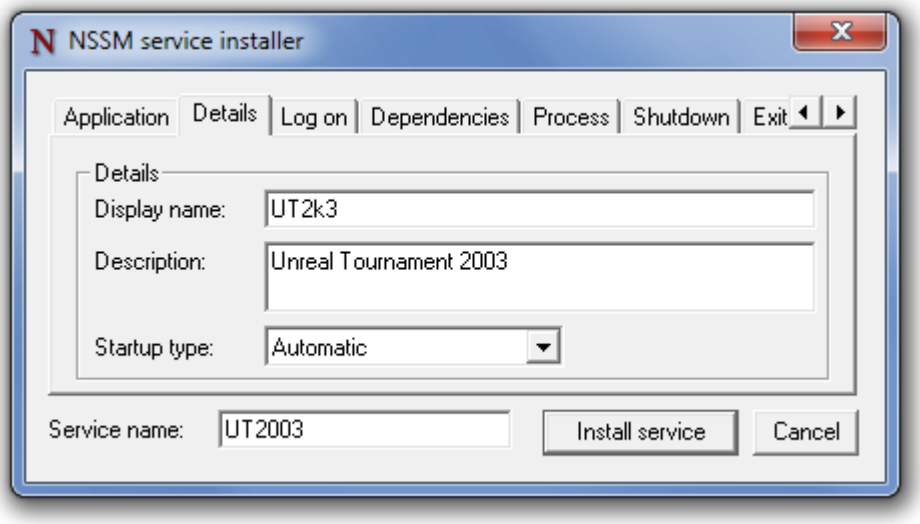
Equivalent commands:

```
nssm set UT2003 Application C:\games\ut2003\System\UCC.exe
nssm set UT2003 AppDirectory C:\games\ut2003\System
nssm set UT2003 AppParameters server
```

Clicking *Install service* completes the installation of the service.

Details tab

The *Details* tab lists system details about the service.

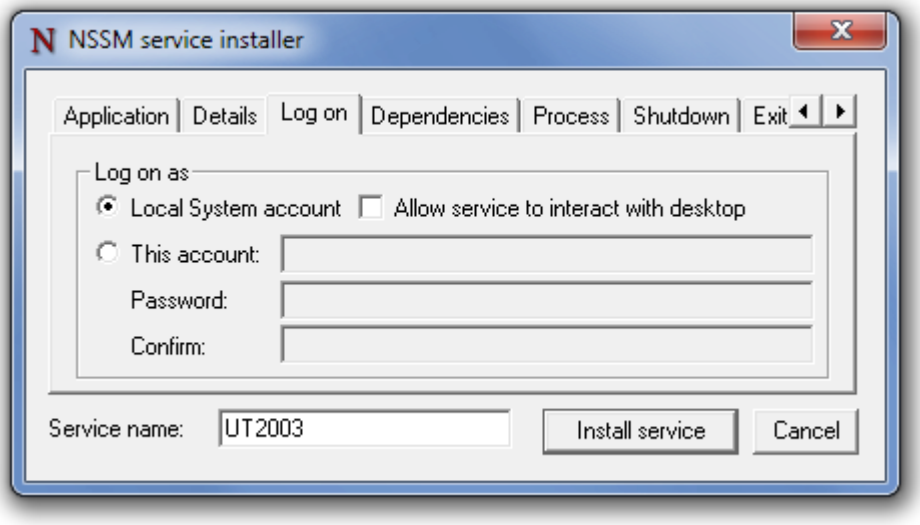


Equivalent commands:

```
nssm set UT2003 DisplayName UT2k3
nssm set UT2003 Description Unreal Tournament 2003
nssm set UT2003 Start SERVICE_AUTO_START
```

Log on tab

The *Log on* tab can be used to manage the user account which will run the service. *nssm* will automatically ensure that the account you choose has the necessary *log on as a service* permissions.



Equivalent commands:

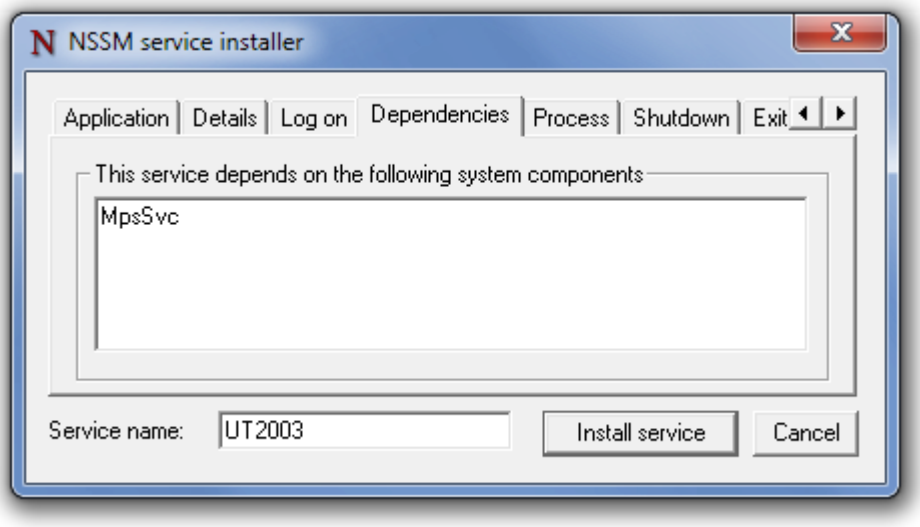
```
nssm set UT2003 ObjectName LocalSystem

nssm set UT2003 Type SERVICE_WIN32_OWN_PROCESS
```

Refer to the command line usage documentation for details on configuring an account and password on the command line. If you need to configure a blank password you *must* use the command line.

Dependencies

The *Dependencies* tab lists any services or service groups which must be started before the the service can run.

You can enter service names or display names, one per line. Service group names must be preceded by the SC_GROUP_IDENTIFIER prefix (the + symbol).



Equivalent commands:

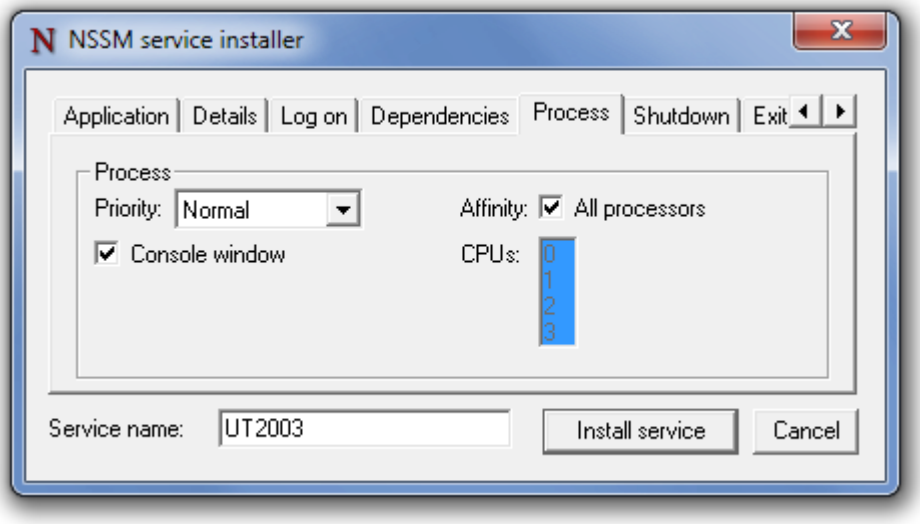
```
nssm set UT2003 DependOnService MpsSvc
```

Process tab

The *Process* tab can be used to set process priority and CPU affinity for the application. By default the application will run with normal priority and be allowed to execute on all CPUs. If you

wish to restrict the process to a subset of available CPUs, uncheck "All processors" and select the CPU(s) as desired.

Process priority and affinity can be changed from the Windows task manager while the service is running.

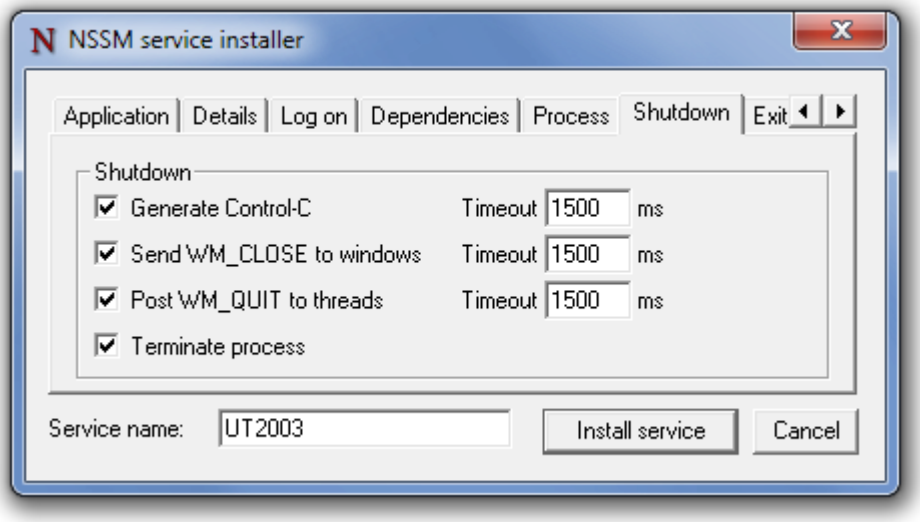


Equivalent commands:

```
nssm set UT2003 AppPriority NORMAL_PRIORITY_CLASS

nssm set UT2003 AppNoConsole 0

nssm set UT2003 AppAffinity All
```

Shutdown tab

The *Shutdown* tab lists the various stop methods and timeouts used when tidying up the application after a crash or when the service is gracefully stopped.



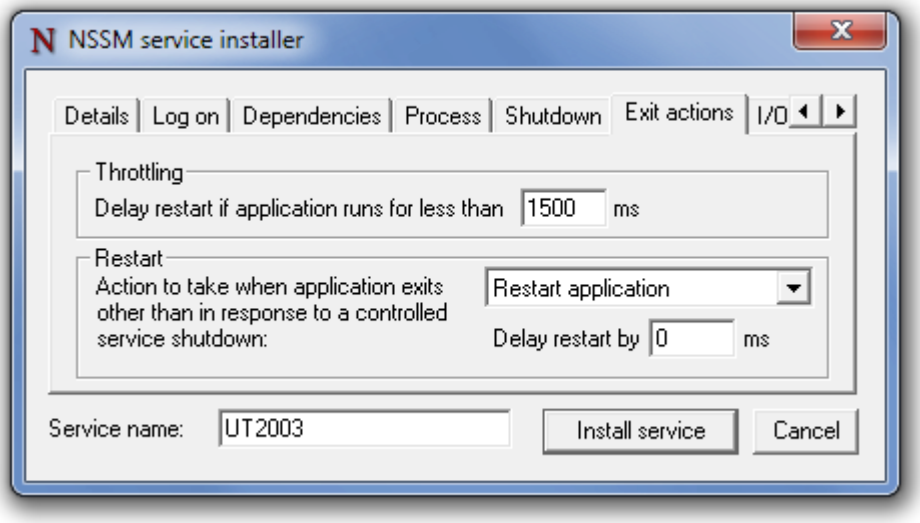
Equivalent commands:

```
nssm set UT2003 AppStopMethodSkip 0

nssm set UT2003 AppStopMethodConsole 1500

nssm set UT2003 AppStopMethodWindow 1500

nssm set UT2003 AppStopMethodThreads 1500
```

Exit actions tab

The *Exit actions* tab can be used to tweak the restart throttling and default action on exit for the service. You can also specify a mandatory delay between automatic restarts of the application.

To configure exit actions for specific application exit codes you must use the registry as described below.



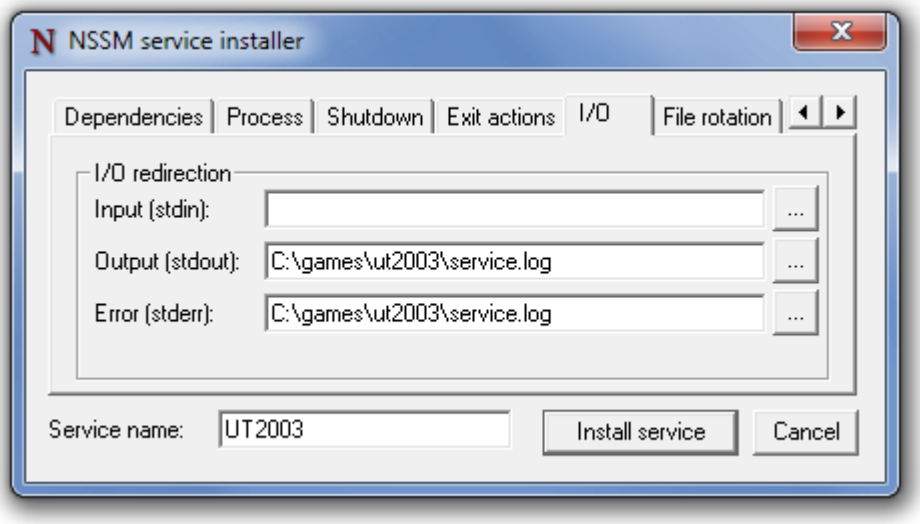
Equivalent commands:

```
nssm set UT2003 AppThrottle 1500

nssm set UT2003 AppExit Default Restart

nssm set UT2003 AppRestartDelay 0
```

I/O tab

The *I/O* tab can be used to specify the input and/or output files used when I/O redirection is enabled. Setting *Output* and *Error* is usually sufficient to capture log messages generated by the application.

Configure I/O in the registry as described below for more control over paths and access modes.



Equivalent commands:

```
nssm set UT2003 AppStdout C:\games\ut2003\service.log
```

```
nssm set UT2003 AppStderr C:\games\ut2003\service.log
```

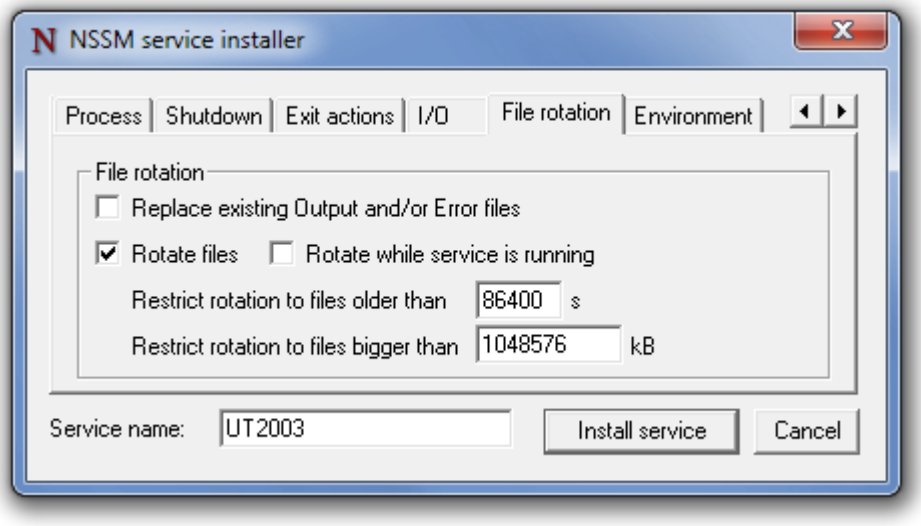
File rotation tab

The *File rotation* tab can be used in conjunction with I/O settings to configure rotation of output files when the service restarts.

If the *Replace existing Output and/or Error files* checkbox is checked, *nssm* will overwrite existing output files when starting the service. The default is to append to any existing files. If the *Rotate files* checkbox is checked, *nssm* will rename existing files prior to setting up I/O redirection. Use the *Restrict rotation* fields to disable file rotation for files which have been modified more recently than the specified number of *seconds* or are smaller than the specified number of *kilobytes*.

By default *nssm* only performs file rotation when the service (re)starts. To enable rotation of files which grow to the specified size limit while the service is running, check the *Rotate while service is running* checkbox. Online rotation ignores any configured file age limit.

Danger, moving parts! Online rotation requires *nssm* to intercept the application's output and write the files itself. Increased complexity necessarily leads to increased risk of failure.

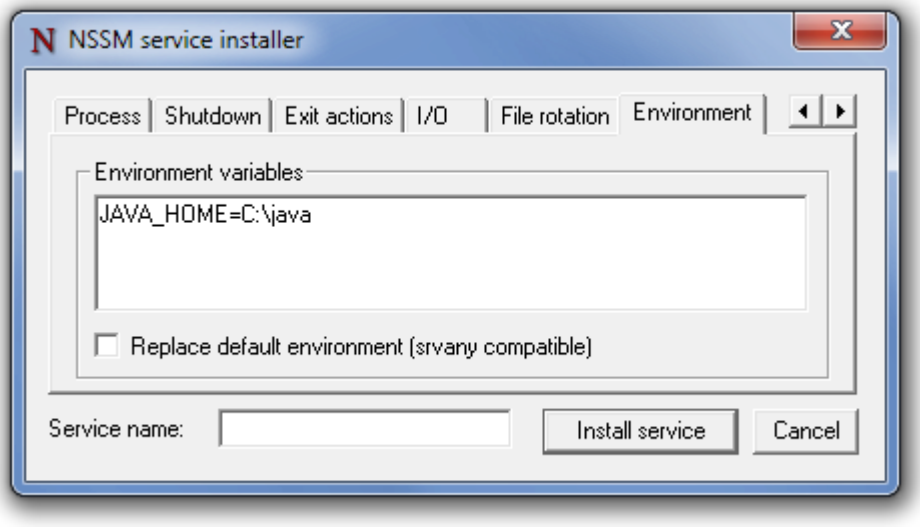


Equivalent commands:

```
nssm set UT2003 AppStdoutCreationDisposition 4

nssm set UT2003 AppStderrCreationDisposition 4

nssm set UT2003 AppRotateFiles 1

nssm set UT2003 AppRotateOnline 0

nssm set UT2003 AppRotateSeconds 86400

nssm set UT2003 AppRotateBytes 1048576
```

Environment tab

The *Environment* tab can be used to specify a newline-separated list of environment variables to pass to the application. If the *Replace default environment* checkbox is checked the variables specified will be the *only* ones passed to the service. When it is left unchecked (the default), the environment created at service startup will be preserved.



Equivalent commands:

```
nssm set <servicename> AppEnvironmentExtra JAVA_HOME=C:\java
```

Installing from the command line

As of version 2.0 you can also bypass the GUI and install a service from the command line. The syntax is:

```
nssm install <servicename> <application> [<options>]
```

Please note that the actual program entered into the services database is *nssm* itself so you must not move or delete *nssm.exe* after installing a service. If you do wish to change the path to *nssm.exe* you can either remove and reinstall the service or edit **HKLM\System\CurrentControlSet\Services*servicename*\ImagePath** to reflect the new location.

Quoting issues

nssm correctly handles paths with spaces but passing arguments to it can be tricky because of how the command prompt works.

If the path to the application contains spaces you will need to enclose it in quotes otherwise the command prompt will interpret the path as *two* arguments.

```
nssm install <servicename> "C:\Program Files\app.exe"
```

If one of the options you wish to provide contains spaces, you will need to quote that too *and* quote the quotation marks themselves.

```
nssm install <servicename> <application> """This is one argument"""
```

Isaballa Sanfelipo suggests a method of installing a Java application from a batch file.

```
nssm install solr "%JavaExe%" -Dsolr.solr.home="\"%CD%\solr"\"
-Djetty.home="\"%CD%"\" -Djetty.logs="\"%CD%\logs"\" -cp
"\"%CD%\lib\*.jar"\";"\"%CD%\start.jar"\" -jar "\"%CD%\start.jar"\"
```

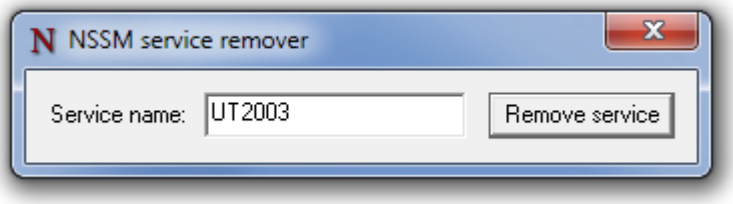
John Duffy needed to pass quotes through to the parameter list.

```
nssm set NodeServer3000 AppParameters """""""$Env:NODE_JS_NPM"""""" start"
```

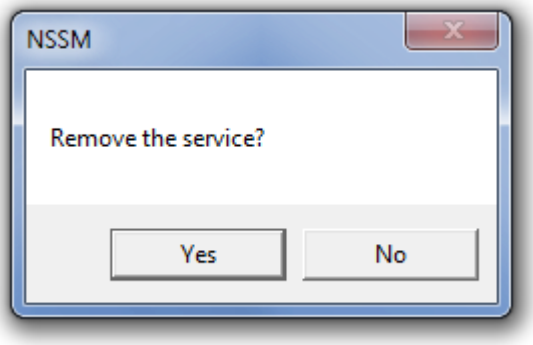
Removing a service

The command to remove a service is:

```
nssm remove <servicename>
```



A confirmation window is displayed before the service is removed.



As of version 2.0 you can also remove services from the command line viz:

```
nssm remove <servicename> confirm
```

nssm will happily try to remove any service, not just ones *nssm* itself manages. Try not to delete services you shouldn't...

Service shutdown

When *nssm* receives a stop command from the Windows service manager, or when it detects that the monitored application has exited, it tries to shut down the monitored application, and any subprocesses, gracefully. If the application's process tree does not exit promptly, *nssm* can forcibly terminate all processes and subprocesses belonging to the application.

There are four stages which *nssm* can use to shut down the application, and by default it will attempt all four in order. It is possible (though not recommended) to disable some or all of the methods from being used. Different applications will respond differently to the various requests, so leaving them all enabled is usually the best way to ensure that the application shuts down gracefully.

First *nssm* will attempt to generate a Control-C event and send it to the application's console. Batch scripts or console applications may intercept the event and shut themselves down gracefully. Java applications tend to respond well to Control-C events. GUI applications do not have consoles and will not respond to this method. *Not supported on Windows 2000.*

Secondly *nssm* will enumerate all windows created by the application and send them a *WM_CLOSE* message. Applications may follow the convention of responding to the message by initiating a graceful exit.

Thirdly *nssm* will enumerate all threads created by the application and send them a *WM_QUIT* message, which will be received if the application has a thread message queue.

As a last resort *nssm* can call `TerminateProcess()` to request that the operating system forcibly terminate the application. The `TerminateProcess()` call cannot be trapped or ignored, so in most

circumstances the application will be killed. However, it is unlikely that it will be able to perform any cleanup operations before it exits.

To disable any of the methods above, create an integer (REG_DWORD) value **HKLM\System\CurrentControlSet\Services*servicename*\Parameters\AppStopMethodSkip** and set it to the sum of one or more of the numbers below.

- **1** - Don't send Control-C to the console.
- **2** - Don't send WM_CLOSE to windows.
- **4** - Don't send WM_QUIT to threads.
- **8** - Don't call TerminateProcess().

If, for example, you knew that an application did not respond to Control-C and did not have a thread message queue, you could set *AppStopMethodSkip* to 5.

It is highly recommended **not** to disable the `TerminateProcess()` call. When the service is stopped *nssm* will exit. If the application is not terminated before that happens it may continue running and *nssm* will no longer be able to control it.

By default *nssm* will wait up to 1500 milliseconds for the application to exit after trying each of the methods described above. The timeout can be configured on a per-method basis by creating integer (REG_DWORD) values under **HKLM\System\CurrentControlSet\Services*servicename*\Parameters** in the registry and setting them to the desired number of milliseconds to wait.

- **AppStopMethodConsole** - Time to wait after sending Control-C.
- **AppStopMethodWindow** - Time to wait after sending WM_CLOSE.
- **AppStopMethodThreads** - Time to wait after sending WM_QUIT.

Note that the timeout applies to all processes spawned by the application so the total timeout may be longer than expected if the application has multiple subprocesses.

Actions on exit

To configure the action which *nssm* should take when the application exits, edit the default value of the key **HKLM\System\CurrentControlSet\Services*servicename*\Parameters\AppExit**. If the key does not exist in the registry when *nssm* runs it will create it and set the value to **Restart**. Change it to either **Ignore** or **Exit** to specify the action taken. *nssm* will only create this key if it doesn't already exist. Your changes will not be overridden.

To specify a different action for particular exit codes, create a string (REG_SZ) value underneath the **AppExit** key whose name is the exit code being considered. For example to stop the service on an exit code of 0 (which usually means the application finished successfully), create **HKLM\System\CurrentControlSet\Services*servicename*\Parameters\AppExit\0** and set it to **Exit**. Look in the Event Log for messages from *nssm* to see what exit codes are returned by your application.

If your application's exit code does not correspond to a registry entry, *nssm* will use the default value of **AppExit** when deciding what to do.

Restart delay

As of version 2.22, *nssm* can apply a mandatory delay between restarts of the application. This could be used, for example, to run a command at regular intervals such as an hourly batch script.

To specify a restart delay, create an integer (REG_DWORD) value **HKLM\System\CurrentControlSet\Services*servicename*\Parameters\AppRestartDelay** and set it to the number of milliseconds to wait between restarts.

The service will report its state as Paused while waiting for the next restart. Sending it a Continue control will temporarily cancel the delay and trigger a restart immediately.

Please see the section on restart throttling below for notes on how *nssm* works when both throttling and restart delays are configured.

Restart throttling

To avoid a tight CPU loop, *nssm* will throttle restarts of the service if the monitored application exits too soon after starting. By default a threshold of 1500 milliseconds is used. To specify a different value, create an integer (REG_DWORD) value **HKLM\System\CurrentControlSet\Services*servicename*\Parameters\AppThrottle** and set it to the required number of milliseconds.

The first restart will be attempted with no delay. If the restarted application continues to exit before running for the threshold amount of milliseconds, *nssm* will pause for at least 2000 milliseconds, doubling the pause time for each subsequent failure. The maximum time it will pause is 256000 milliseconds, around four minutes. The delay counter is reset when the service successfully runs for at least the threshold time.

If you determine why the service failed and take action to correct the problem, you can send a Continue control to the service, which will be shown as Paused. In this way you can avoid having to wait for the next restart attempt.

When a restart delay is configured and the application exits prematurely, *nssm* will throttle the restart by whichever is *longer* of the configured delay and calculated throttle time. If, for instance, you configured a restart delay of 3000 milliseconds and the service failed on each startup, the first restart attempt would be delayed by 3000 milliseconds because 3000ms configured is longer than 0ms from throttling. The second attempt would also be delayed by 3000 milliseconds; 3000ms configured is again longer than 2000ms from throttling. The third attempt would be delayed by 4000 milliseconds; longer than the configured 3000ms.

For this reason, if you intend to use a restart delay to configure a short-running service which repeats at an interval less than five minutes, you should consider lowering the value of **AppThrottle**.

Process priority and CPU affinity

As of version 2.22, *nssm* can manage the CPU affinity and process priority of the managed application.

By default application will be launched with normal process priority and be allowed to execute on any CPU. *nssm* will look under **HKLM\System\CurrentControlSet\Services*servicename*\Parameters** for registry entries to configure the application startup.

If the integer (REG_DWORD) value **AppPriority** is set, *nssm* will interpret its value as an argument to `SetPriorityClass()` and start the application with the specified priority.

If the string (REG_SZ) value **AppAffinity** is set, *nssm* will interpret it as a comma-separated list of CPU IDs, starting from *0* on which the application can run. Alternatively, a range of IDs may be specified by separating the indices with a dash.

Only digits, dashes and commas are valid in an affinity string.

For example, the the string *0-2,4* specifies that the application may run on the first, second, third and fifth CPUs in the system.

Console window

As of version 2.22, *nssm* will by default create a new console window for the application. This allows some programs to work which would otherwise fail, such as those which expect to be able to read user input. The console window can be disabled if it is not needed by setting the integer (REG_DWORD) value **AppNoConsole** under **HKLM\System\CurrentControlSet\Services*servicename*\Parameters** to a non-zero value.

I/O redirection

nssm can redirect the managed application's I/O to any path capable of being opened by `CreateFile()`. This feature may be useful if your application expects to be able to log to the console.

nssm will look under **HKLM\System\CurrentControlSet\Services*servicename*\Parameters** for keys corresponding to arguments to `CreateFile()`. All are optional. If no path is given for a particular stream it will not be redirected. If a path is given but any of the other values they will receive sensible defaults.

- **AppStdin** (string) - Path to receive input.
- **AppStdinShareMode** (integer) - `ShareMode` argument for the input.
- **AppStdinCreationDisposition** (integer) - `CreationDisposition` argument for the input.
- **AppStdinFlagsAndAttributes** (integer) - `FlagsAndAttributes` argument for the input.
- **AppStdout** (string) - Path to receive output.
- **AppStdoutShareMode** (integer) - `ShareMode` argument for the output.
- **AppStdoutCreationDisposition** (integer) - `CreationDisposition` argument for the output.
- **AppStdoutFlagsAndAttributes** (integer) - `FlagsAndAttributes` argument for the output.
- **AppStderr** (string) - Path to receive error output.
- **AppStderrShareMode** (integer) - `ShareMode` argument for the error output.
- **AppStderrCreationDisposition** (integer) - `CreationDisposition` argument for the error output.
- **AppStderrFlagsAndAttributes** (integer) - `FlagsAndAttributes` argument for the error output.

In general it is advisable to set both **AppStdout** and **AppStderr** in order to log output, as applications may log informational and error messages separately.

It is possible to direct both stderr and stdout to the same path but due to a limitation with *nssm* you must provide the *exact* same string in both the **AppStdout** and **AppStderr** registry values. Only if the two entries are *the same* will *nssm* be able to interleave the two streams.

File rotation

As of version 2.22, if I/O redirection is enabled, *nssm* can rotate existing output files prior to launching the application. *nssm* can also rotate files while the service is running. See online

rotation below.

To enable rotation, create an integer (REG_DWORD) value **HKLM\System\CurrentControlSet\Services*servicename*\Parameters\AppRotate** and set it to 1. Before (re)starting the service, *nssm* will rotate the file(s) configured in **AppStdout** and/or **AppStderr** if they already exist.

Existing files will be renamed according to a template which uses the last modification time of the file. For example, *C:\Services\myservice.log* might be rotated to *C:\Services\myservice-20140114T180840.953.log*.

Note that the timestamp is in ISO8601 format, so a list of rotated files sorted by name will show the oldest file first, and that it includes a millisecond part, so that files will not be lost if the service exits and is restarted in less than a second.

Two additional registry settings under **HKLM\System\CurrentControlSet\Services*servicename*\Parameters** can be used to tweak how *nssm* rotates files.

If the integer (REG_DWORD) value **AppRotateSeconds** is set, *nssm* will not rotate any file which was last modified *less* than the configured number of seconds ago.

If the integer (REG_DWORD) value **AppRotateBytes** is set, *nssm* will not rotate any file which is *smaller* than the configured number of bytes. *nssm* can also handle files whose size is too large to be expressed in 32-bits, in case you decide that it's fine for a log file to grow to 4GB in size but any more is just *too big*. The value of **AppRotateBytesHigh** will be interpreted as the high order part of a 64-bit size.

If both **AppRotateSeconds** and **AppRotateBytes(High)** are set, *nssm* will require that both criteria are met for a file to be rotated.

Online rotation

If the integer (REG_DWORD) value **AppRotateOnline** is set to 1, *nssm* can rotate files which grow to the configured file size limit while the service is running. The value of **AppRotateSeconds** is ignored for the purposes of online rotation, though it will still apply for rotation before the service (re)starts.

AppRotateOnline is ignored if **AppRotate** is not set.

When online rotation is enabled, *nssm* reads the application's stdout and/or stderr and writes the output files itself. Doing so introduces a degree of complexity compared to simple I/O redirection, so should not be used for services which do not require it. Although *nssm* will try to handle I/O errors gracefully, if something goes wrong it is possible for output from the application to be lost until the service is restarted. See the technical discussion for more details about how *nssm* handles I/O redirection.

On-demand rotation

nssm can rotate output files on-demand, regardless of whether or not they have hit the configured size limit. To request file rotation for a service, send user-defined service control 128 or run the command:

```
nssm rotate <servicename>
```

A limitation of on-demand rotation is that the actual file renaming won't happen until after the next line of input has been read from the application. For that reason there may be a considerable delay

between issuing the rotation request and the rotation taking place, depending on how verbose the application is.

On-demand rotation will work even if **AppRotateBytes** is not configured, ie if the service would not rotate files while running. It will not work, however, unless both **AppRotate** and **AppRotateOnline** are configured.

I/O redirection technical details

There are three cases to consider when looking at how a service handles I/O.

No redirection

In the simplest case, *nssm* is not configured with any I/O redirection. It will launch the application with stdin, stdout and stderr connected to a console instance. If the service runs under the LOCALSYSTEM account and is configured to interact with the desktop, you may be able to view the output directly.

I/O redirected; online rotation disabled

If stdout and/or stderr are redirected and online rotation is disabled, *nssm* will call `CreateFile()` to open a handle for each I/O stream, then call `DuplicateHandle()` to set a copy of the handle in the `STARTUPINFO` datastructure passed to `CreateProcess()`. Thus the application will run with an open handle to the file whereas *nssm* itself has no open handles.

I/O redirected; online rotation enabled

This case is by far the most complex of the three and is thus necessarily most at risk of something going wrong, potentially resulting in the loss of output from the application.

nssm first calls `CreateFile()` just as in the previous case. It then calls `CreatePipe()` to open an anonymous pipe. The reading end of the pipe and the handle to the output file are passed to a new thread which does the actual writing. The writing end of the pipe is duplicated with `DuplicateHandle()` for passing to `CreateProcess()`. Thus the application will run with an open handle to one end of the pipe and *nssm* will run with a handle to the other end of the pipe and a handle to the output file.

The writing thread runs a simple loop wherein it reads data from the pipe, ie the application's output, into a buffer with `ReadFile()` then writes the contents of that buffer to the output file with `WriteFile()`. If the file hits the configured size limit the thread will write up to the next newline, close its output handle, rotate the file and open a new handle to continue writing to the new file.

When *nssm* receives an on-demand rotation request it will set a flag which instructs the writing thread to perform a rotation after the next `ReadFile()` call completes, regardless of the file size. Because `ReadFile()` blocks execution until something is read, *nssm* won't actually get round to rotating the file until the application produces the next line of output.

nssm doesn't know whether the output from the application will be Unicode or ANSI, so prior to writing the first data to the file, or to rotating, it calls `IsTextUnicode()` to try to determine the text encoding that's in use. If it (looks like it) is Unicode, *nssm* writes a UTF-16 byte-order marker to the beginning of the new file so that it will be read correctly when opened with a text editor.

If both stdout and stderr are redirected, and to separate files, *nssm* will spawn a writing thread for each. *nssm* will deal with an application which writes stdout in Unicode and stderr in ANSI - or vice versa - if you are unfortunate (or evil) enough to run one.

It should be clear from the above that there are a number of pitfalls to online rotation. *nssm* will try to deal gracefully with any problems during the I/O process but to be safe you should consider not using online rotation unless your output is so voluminous that you have no choice.

Environment variables

As of version 2.11, *nssm* respects the **AppEnvironment** registry value supported by *srvany*. To specify a list of environment variables to pass to the monitored application, create a multi-valued string (REG_MULTI_SZ) value **HKLM\System\CurrentControlSet\Services*servicename*\Parameters\AppEnvironment** where each entry is of the form *KEY*=*VALUE*.

It is possible to omit the *VALUE* if you just want the environment variable *KEY* to exist but the = symbol is mandatory. *The service will not run if an invalid environment is specified!*

As of version 2.19, *nssm* also respects the **AppEnvironmentExtra** registry value, which should have the same format as **AppEnvironment**. Environment variables set in **AppEnvironmentExtra** will be *added* to the service's default environment.

For compatibility with *srvany*, the environment variables specified in **AppEnvironment** will *replace* those set by the system at service startup. Since that probably isn't what you want, use **AppEnvironmentExtra** instead.